



Chapter 16: Query Optimization

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Outline

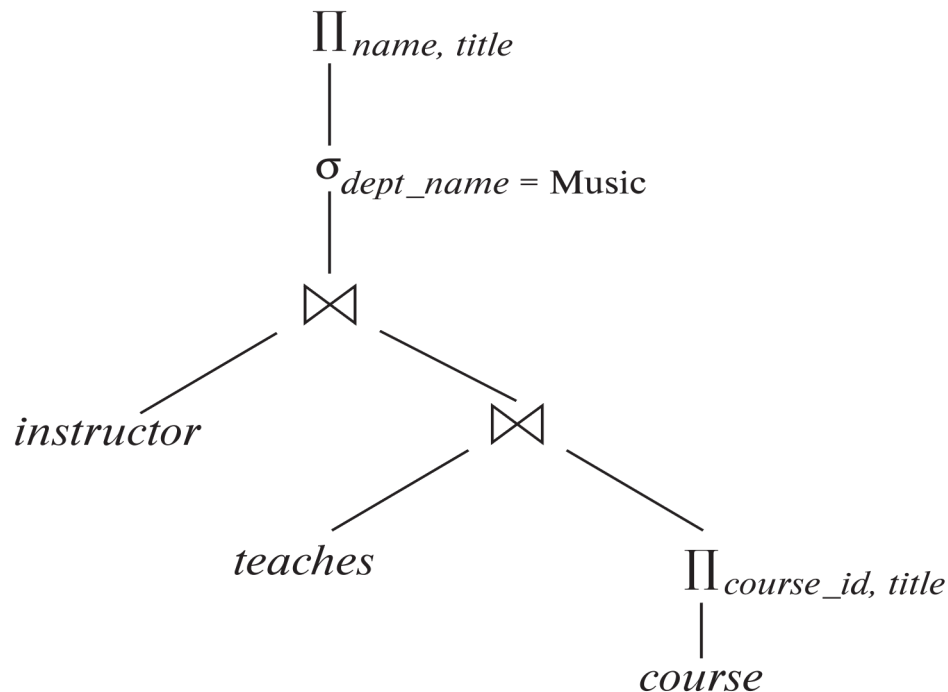
- Introduction
- Transformation of Relational Expressions
- Catalog Information for Cost Estimation
- Statistical Information for Cost Estimation
- Cost-based optimization
- Dynamic Programming for Choosing Evaluation Plans
- Materialized views



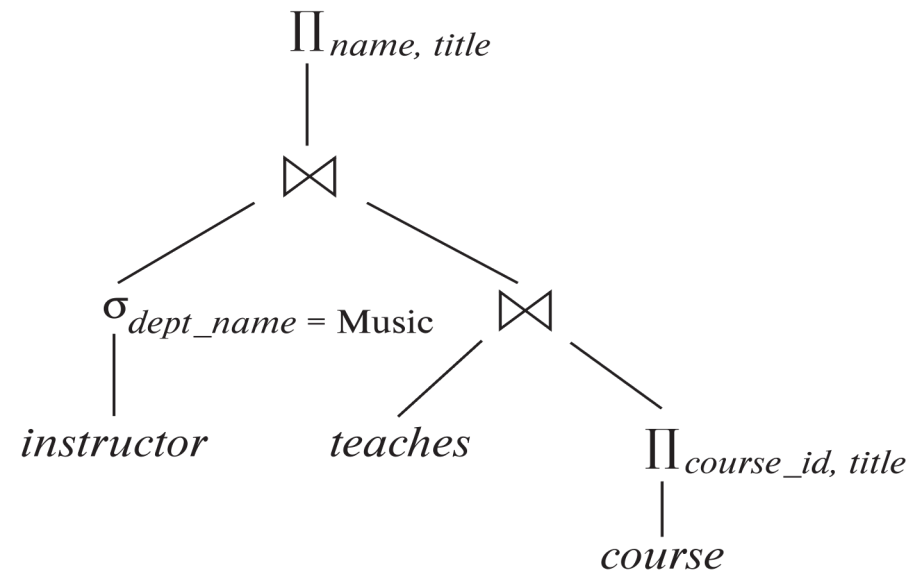
Introduction

- Alternative ways of evaluating a given query
 - Equivalent expressions

$\Pi_{name, title} (\sigma_{dept_name = \text{"Music"}} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$



(a) Initial expression tree



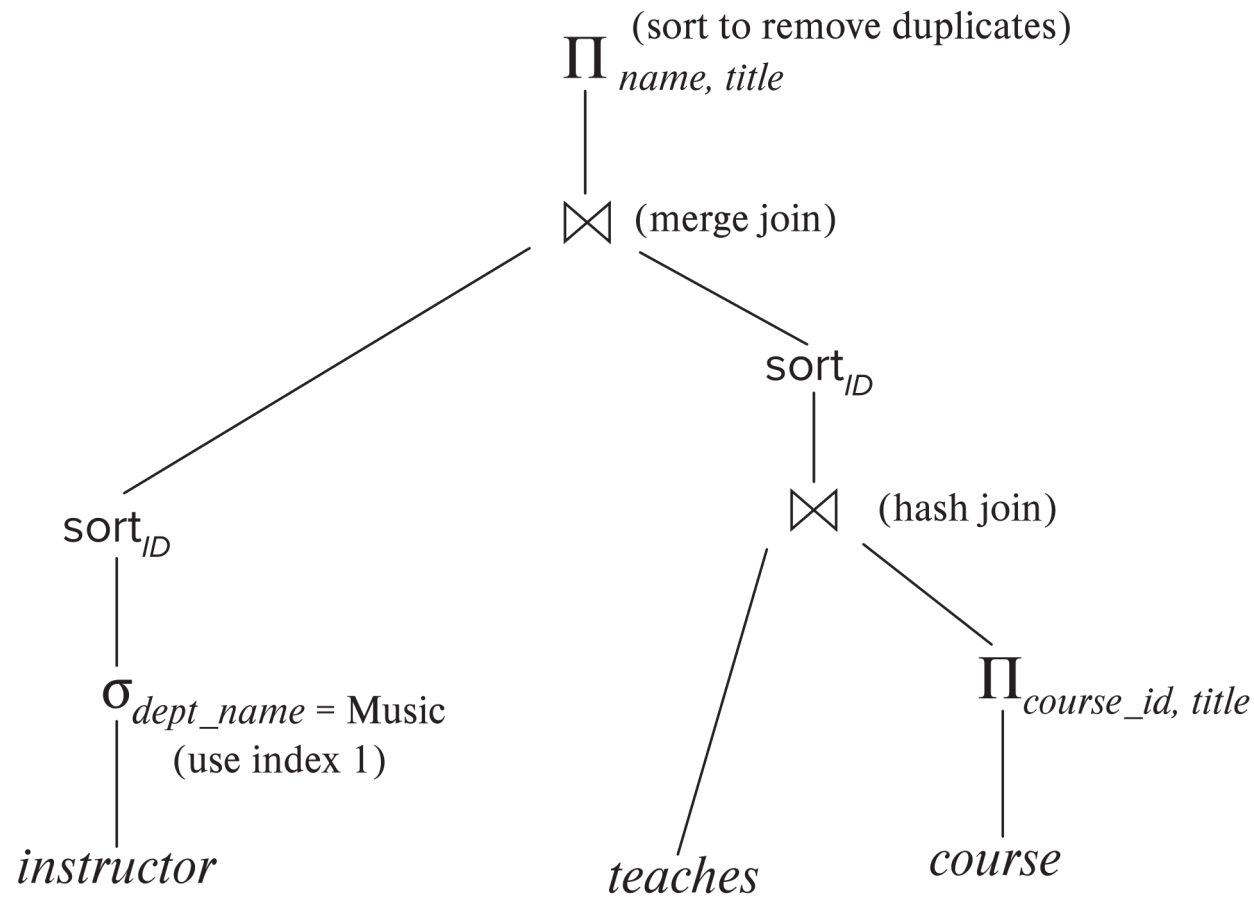
(b) Transformed expression tree

$\Pi_{name, title} ((\sigma_{dept_name = \text{"Music"}} (instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$



Introduction (Cont.)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.





Introduction (Cont.)

- Cost difference between evaluation plans for a query can be enormous
 - E.g., seconds vs. days in some cases
- Steps in **cost-based query optimization**
 1. Generate logically equivalent expressions using **equivalence rules**
 2. Annotate resultant expressions to get alternative query plans
 3. Choose the cheapest plan based on **estimated cost**
- Estimation of plan cost based on:
 - Statistical information about relations. Examples:
 - number of tuples, number of distinct values for an attribute
 - Statistics estimation for intermediate results
 - to compute cost of complex expressions
 - Cost formulae for algorithms, computed using statistics



Viewing Query Evaluation Plans

- Most database support **explain** <query>
 - Displays plan chosen by query optimizer, along with cost estimates
 - Some syntax variations between databases
 - Oracle: **explain plan for** <query> followed by **select * from** table (*dbms_xplan.display*)
 - SQL Server: **set showplan_text on**
- Some databases (e.g. PostgreSQL) support **explain analyse** <query>
 - Shows actual runtime statistics found by running the query, in addition to showing the plan
- Some databases (e.g. PostgreSQL) show cost as *f..l*
 - *f* is the cost of delivering first tuple and *l* is cost of delivering all results



Generating Equivalent Expressions

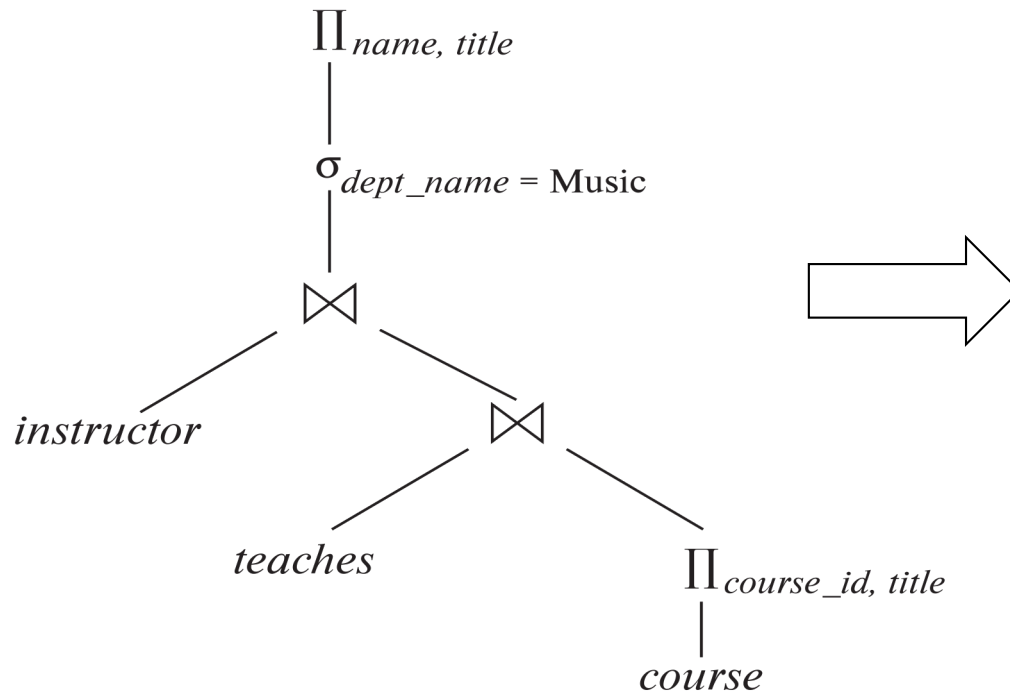


Transformation of Relational Expressions

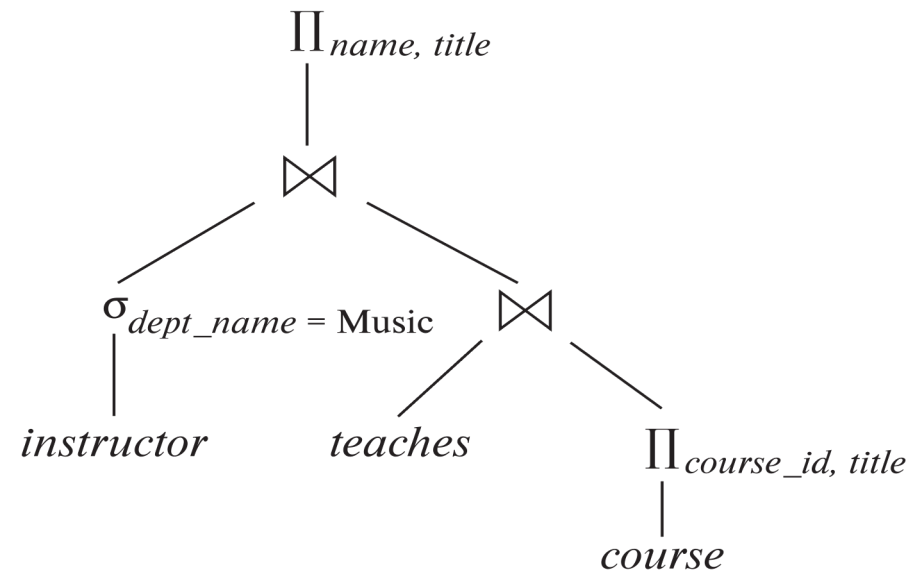
- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
 - Note: order of tuples is irrelevant
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of the first form by the second, or vice versa



Transformation of Relational Expressions

$$\Pi_{name, title} (\sigma_{dept_name = \text{"Music"}} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$


(a) Initial expression tree



(b) Transformed expression tree

$$\Pi_{name, title} ((\sigma_{dept_name = \text{"Music"}} (instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$$



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E)$$

where $L_1 \subseteq L_2 \dots \subseteq L_n$

4. Selections can be combined with Cartesian products and theta joins.

a. $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$

b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$



Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .



Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:

(a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

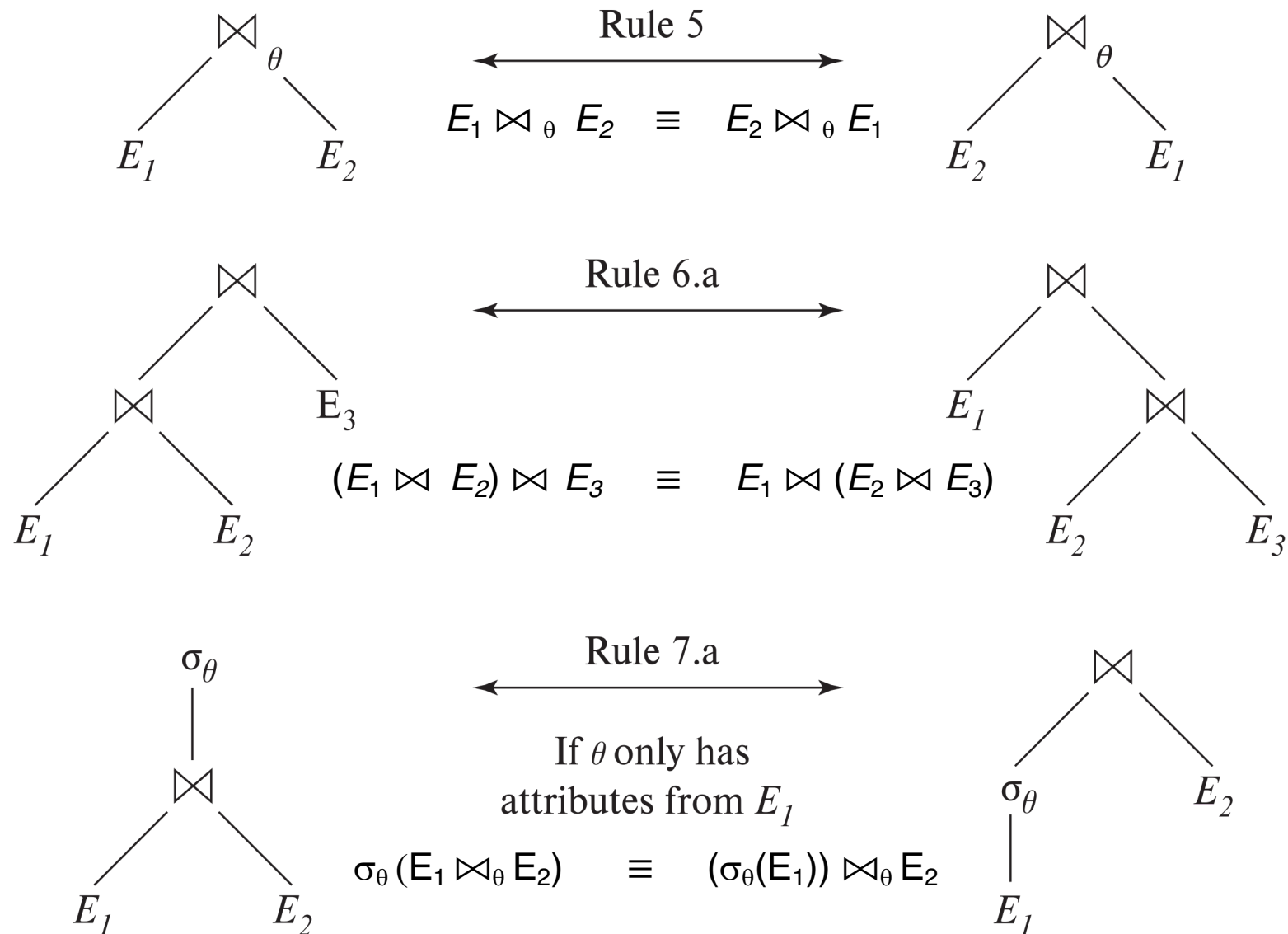
$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

(b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$



Pictorial Depiction of Equivalence Rules





Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

(a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1}(E_1) \bowtie_{\theta} \Pi_{L_2}(E_2)$$

(b) In general, consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- Let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_{\theta} \Pi_{L_2 \cup L_4}(E_2))$$



Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 \equiv E_2 \cup E_1$$

$$E_1 \cap E_2 \equiv E_2 \cap E_1$$

(set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\text{a. } \sigma_{\theta}(E_1 \cup E_2) \equiv \sigma_{\theta}(E_1) \cup \sigma_{\theta}(E_2)$$

$$\text{b. } \sigma_{\theta}(E_1 \cap E_2) \equiv \sigma_{\theta}(E_1) \cap \sigma_{\theta}(E_2)$$

$$\text{c. } \sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) \equiv (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$



Transformation Example: Pushing Selections

- Performing the selection as early as possible reduces the size of the relation to be joined.
- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach
 - $\Pi_{name, title}(\sigma_{dept_name = 'Music'}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$
- Transformation using rule 7a.
 - $\Pi_{name, title}((\sigma_{dept_name = 'Music'}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$



Example with Multiple Transformations

- Query: Find the names of all instructors in the Music department who have taught a course in 2017, along with the titles of the courses that they taught

- $$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2017} (instructor \bowtie (teaches \bowtie \Pi_{course_id, title} (course))))$$

- Transformation using join associatively (Rule 6a):

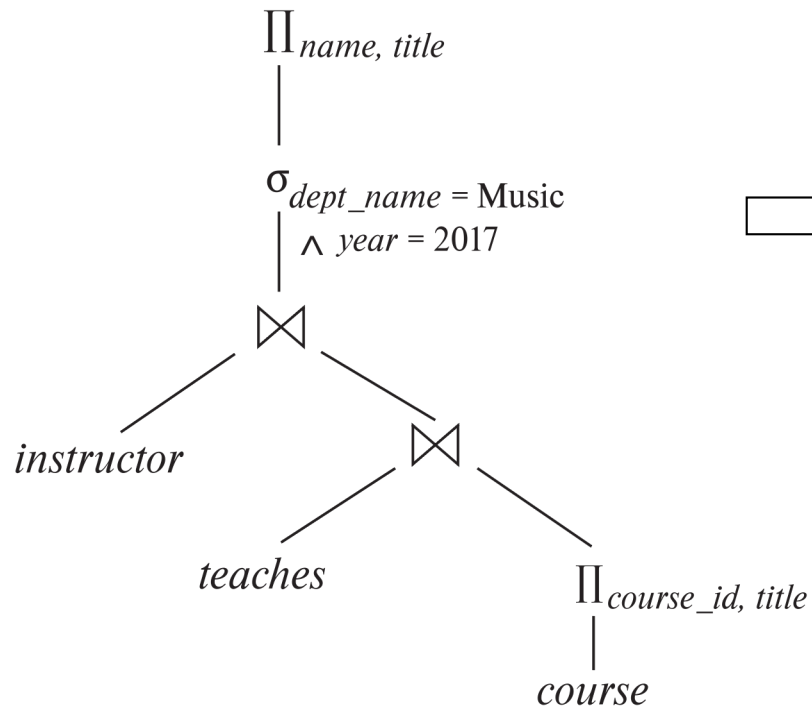
- $$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"} \wedge year = 2017} ((instructor \bowtie teaches) \bowtie \Pi_{course_id, title} (course)))$$

- The second form provides an opportunity to further apply the “perform selections early” rule, resulting in the subexpression

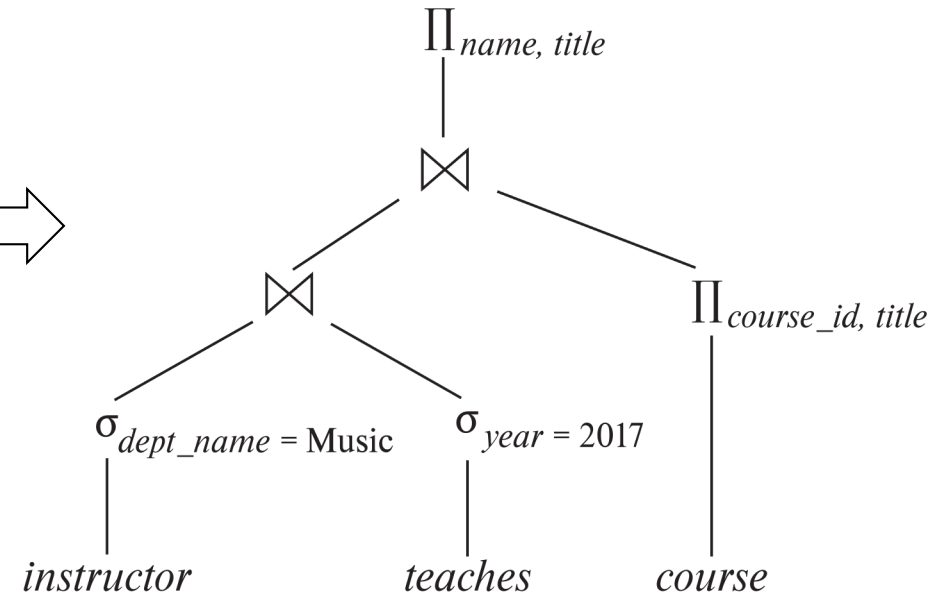
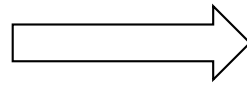
$$\sigma_{dept_name = \text{"Music"}} (instructor) \bowtie \sigma_{year = 2017} (teaches)$$



Multiple Transformations (Cont.)



(a) Initial expression tree



(b) Tree after multiple transformations



Transformation Example: Pushing Projections

- Performing the projection as early as possible reduces the size of the relation to be joined.

- Consider: $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches) \bowtie \Pi_{course_id, title}(course))$

- When we compute

$$\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches)$$

we obtain a relation whose schema is:

$(ID, name, dept_name, salary, course_id, sec_id, semester, year)$

- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = \text{"Music"}}(instructor \bowtie teaches)) \bowtie \Pi_{course_id, title}(course))$$



Join Ordering Example

- For all relations r_1 , r_2 , and r_3 ,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

(Join Associativity) $\bowtie$

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

so that we compute and store a smaller temporary relation.



Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name, title}(\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches \\ \bowtie \Pi_{course_id, title}(course))$$

- Could compute $teaches \bowtie \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{“Music”}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
 - it is better to compute

$$\sigma_{dept_name = \text{“Music”}}(instructor) \bowtie teaches$$

first.



Enumeration of Equivalent Expressions

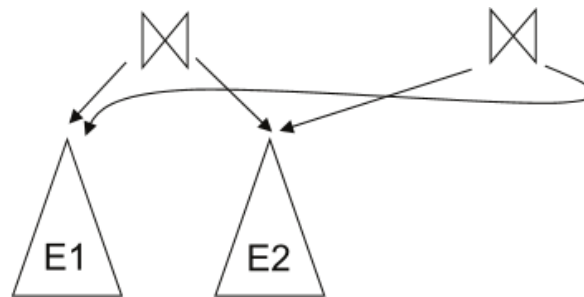
- Query optimizers use equivalence rules to **systematically** generate expressions equivalent to the given expression
- Can generate all equivalent expressions as follows:
 - Repeat
 - apply all applicable equivalence rules on every subexpression of every equivalent expression found so far
 - add newly generated expressions to the set of equivalent expressions

Until no new equivalent expressions are generated above
- The above approach is very expensive in space and time
 - Two approaches
 - Optimized plan generation based on transformation rules
 - Special case approach for queries with only selections, projections and joins



Implementing Transformation Based Optimization

- Space requirements reduced by sharing common sub-expressions:
 - when E1 is generated from E2 by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers
 - E.g., when applying join commutativity



- Same sub-expression may get generated multiple times
 - Detect duplicate sub-expressions and share one copy
- Time requirements are reduced by not generating all expressions
 - Dynamic programming
 - We will study only the special case of dynamic programming for join order optimization



Cost Estimation

- Cost of each operator computer as described in Chapter 15
 - Need statistics of input relations
 - E.g., number of tuples, sizes of tuples
- Inputs can be results of sub-expressions
 - Need to estimate statistics of expression results
 - To do so, we require additional statistics
 - E.g., number of distinct values for an attribute
- More on cost estimation later



Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
 - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
 - merge-join may be more costly than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
 - nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
 1. Search all the plans and choose the best plan in a cost-based fashion.
 2. Uses heuristics to choose a plan.



Cost-Based Optimization

- Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$.
- When $n=3$, there are 12 join orders

$$\begin{array}{cccc}
 r_1 \bowtie (r_2 \bowtie r_3) & r_1 \bowtie (r_3 \bowtie r_2) & (r_2 \bowtie r_3) \bowtie r_1 & (r_3 \bowtie r_2) \bowtie r_1 \\
 r_2 \bowtie (r_1 \bowtie r_3) & r_2 \bowtie (r_3 \bowtie r_1) & (r_1 \bowtie r_3) \bowtie r_2 & (r_3 \bowtie r_1) \bowtie r_2 \\
 r_3 \bowtie (r_1 \bowtie r_2) & r_3 \bowtie (r_2 \bowtie r_1) & (r_1 \bowtie r_2) \bowtie r_3 & (r_2 \bowtie r_1) \bowtie r_3
 \end{array}$$

- There are $(2(n-1))!/(n-1)!$ different join orders for above expression. With $n = 7$, the number is 665280, with $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \dots, r_n\}$ is computed only once and stored for future use.
- $(r_1 \bowtie r_2 \bowtie r_3) \bowtie r_4 \bowtie r_5$
 - There are 12 different join orders for computing $r_1 \bowtie r_2 \bowtie r_3$
 - It will be joined with r_4 and r_5 : another 12 orders
 - $12+12 = 24$



Dynamic Programming in Optimization

- To find best join tree for a set of n relations:
 - To find best plan for a set S of n relations, consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S .
 - Recursively compute costs for joining subsets of S to find the cost of each plan. Choose the cheapest.
 - Base case for recursion: single relation access plan
 - Apply all selections on R_i using best choice of indices on R_i
 - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it
 - Dynamic programming



Join Order Optimization Algorithm

```
procedure findbestplan(S)
  if (bestplan[S].cost  $\neq \infty$ )
    return bestplan[S]
  // else bestplan[S] has not been computed earlier, compute it now
  if (S contains only 1 relation)
    set bestplan[S].plan and bestplan[S].cost based on the best way
    of accessing S using selections on S and indices (if any) on S
  else for each non-empty subset S1 of S such that S1  $\neq S$ 
    P1 = findbestplan(S1)
    P2 = findbestplan(S - S1)
    for each algorithm A for joining results of P1 and P2
      ... compute plan and cost of using A (see next page) ..
      if cost < bestplan[S].cost
        bestplan[S].cost = cost
        bestplan[S].plan = plan;
  return bestplan[S]
```




Join Order Optimization Algorithm (cont.)

```
for each algorithm  $A$  for joining results of  $P1$  and  $P2$ 
    // For indexed-nested loops join, the outer could be  $P1$  or  $P2$ 
    // Similarly for hash-join, the build relation could be  $P1$  or  $P2$ 
    // We assume the alternatives are considered as separate algorithms
    if algorithm  $A$  is indexed nested loops
        Let  $P_i$  and  $P_o$  denote inner and outer inputs
        if  $P_i$  has a single relation  $r_i$  and  $r_i$  has an index on the join attribute
             $plan =$  “execute  $P_o.plan$ ; join results of  $P_o$  and  $r_i$  using  $A$ ”,
                with any selection conditions on  $P_i$  performed as part of
                the join condition
             $cost = P_o.cost + \text{cost of } A$ 
        else  $cost = \infty$ ; /* cannot use indexed nested loops join */
    else
         $plan =$  “execute  $P1.plan$ ; execute  $P2.plan$ ;
            join results of  $P1$  and  $P2$  using  $A$ ,”
         $cost = P1.cost + P2.cost + \text{cost of } A$ 
.... See previous page
```



Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
 - Perform selection early (reduces the number of tuples)
 - Perform projection early (reduces the number of attributes)
 - Perform most restrictive selection and join operations (i.e., with smallest result size) before other similar operations.
 - Some systems use only heuristics, others combine heuristics with partial cost-based optimization.



Statistics for Cost Estimation



Statistical Information for Cost Estimation

- n_r : number of tuples in a relation r .
- l_r : size of a tuple of r .
- f_r : number of tuples of r that can fit into one block.
- b_r : number of blocks containing tuples of r .
- If the tuples of r are stored together physically in a file, then:

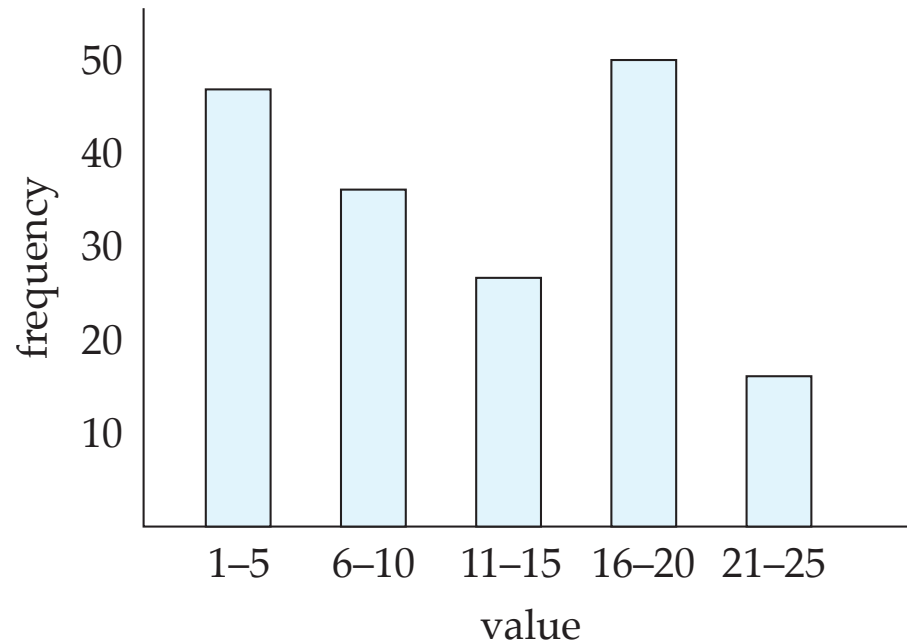
$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

- $V(A, r)$: number of distinct values for attribute A in r ; same as the size of $\Pi_A(r)$.



Histograms

- Histogram on attribute *age* of relation *person*



- **Equi-width** histograms
- **Equi-depth** histograms break up range such that each range has (approximately) the same number of tuples
 - E.g. (4, 8, 14, 19)
- Many databases also store n **most-frequent values** and their counts
 - Histogram is built on remaining values only



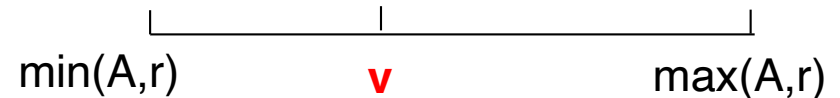
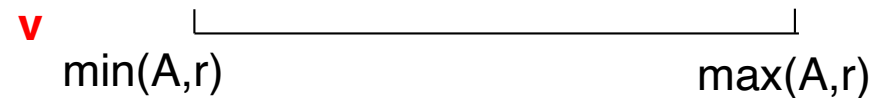
Histograms (cont.)

- Histograms and other statistics usually computed based on a **random sample**
- Statistics may be out of date
 - Some database require a **analyze** command to be executed to update statistics
 - Others automatically recompute statistics
 - e.g., when number of tuples in a relation changes by some percentage



Selection Size Estimation

- $\sigma_{A=v}(r)$
 - $n_r / V(A,r)$: number of records that will satisfy the selection condition
 - Equality condition on a key attribute: *size estimate* = 1
- $\sigma_{A \leq v}(r)$ (the case of $\sigma_{A \geq v}(r)$ is symmetric)
 - Let c denote the estimated number of tuples satisfying the condition.
 - If $\min(A,r)$ and $\max(A,r)$ are available in catalog
 - $c = 0$ if $v < \min(A,r)$
 - $c = n_r \cdot \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$





Size Estimation of Complex Selections

- The **selectivity** of a condition θ_i is the probability that a tuple in the relation r satisfies θ_i .
 - If s_i is the number of satisfying tuples in r , the selectivity of θ_i is given by s_i / n_r .

- **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$. Assuming independence, estimate of tuples in the result is:

$$n_r * \frac{s_1 * s_2 * \dots * s_n}{n_r^n}$$

- **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$. Estimated number of tuples:

$$n_r * \left(1 - \left(1 - \frac{s_1}{n_r} \right) * \left(1 - \frac{s_2}{n_r} \right) * \dots * \left(1 - \frac{s_n}{n_r} \right) \right)$$

↙ The probability that the tuple will satisfy the disjunction

- **Negation:** $\sigma_{\neg \theta}(r)$. Estimated number of tuples:

$$n_r - \text{size}(\sigma_{\theta}(r))$$



Join Operation: Running Example

Running example:

$student \bowtie takes$

Catalog information for the join example:

- $n_{student} = 5,000$.
- $f_{student} = 50$, implying that $b_{student} = 5000/50 = 100$ blocks.
- $n_{takes} = 10000$.
- $f_{takes} = 25$, which implies that $b_{takes} = 10000/25 = 400$ blocks.
- $V(ID, student) = 5000$ (*primary key!*)
- $V(ID, takes) = 2500$ (# of distinct values of ID in *takes*), implying that on average, each student (who has taken a course) takes 4 courses.
 - Attribute *ID* in *takes* is a foreign key referencing *student*.



Estimation of the Size of Joins

- The Cartesian product $r \times s$ contains $n_r \cdot n_s$ tuples.
- If $R \cap S = \emptyset$, then $r \bowtie s$ is the same as $r \times s$.
- If $R \cap S$ is a key for R , then a tuple of s will join with at most one tuple from r
 - therefore, the number of tuples in $r \bowtie s$ is no greater than the number of tuples in s .
- If $R \cap S$ is a foreign key in S referencing R , then the number of tuples in $r \bowtie s$ is exactly the same as the number of tuples in s .
 - The case for $R \cap S$ being a foreign key referencing S is symmetric.



Estimation of the Size of Joins

$$n_{student} = 5,000.$$

$$n_{takes} = 10000.$$

- In the example query $student \bowtie takes$, ID in $takes$ is a foreign key referencing $student$
 - hence, the result has exactly n_{takes} tuples, which is 10000



Estimation of the Size of Joins (Cont.)

- If $R \cap S = \{A\}$ is not a key for R or S .
If we assume that every tuple t in R produces tuples in $R \bowtie S$. For each t , it produces tuples in $R \bowtie S$: $\frac{n_s}{V(A, s)}$

Then considering all tuples in R , the estimate is:

$$\frac{n_r * n_s}{V(A, s)}$$

If the reverse is true, the estimate obtained will be:

$$\frac{n_r * n_s}{V(A, r)}$$

The lower of these two estimates is probably the more accurate one.



Estimation of the Size of Joins (Cont.)

- Compute the size estimates for *student* ⋈ *takes* without using information about foreign keys:
 - $V(ID, student) = 5000$ $n_{student} = 5,000.$
 - $V(ID, takes) = 2500,$ $n_{takes} = 10000.$
 - The two estimates are $5000 * 10000/2500 = 20,000$ and $10000 * 5000/5000 = 10000$
 - We choose the lower estimate, which in this case, is the same as our earlier computation using foreign keys (each student ID in *takes* produce tuples in the join).



Size Estimation for Other Operations

- Projection: estimated size of $\Pi_A(r) = V(A, r)$
- Set operations
 - For unions/intersections of selections on the same relation: rewrite and use size estimate for selections
 - E.g., $\sigma_{\theta_1}(r) \cup \sigma_{\theta_2}(r)$ can be rewritten as $\sigma_{\theta_1 \text{ or } \theta_2}(r)$
 - For operations on different relations:
 - estimated size of $r \cup s = \text{size of } r + \text{size of } s$.
 - estimated size of $r \cap s = \text{minimum size of } r \text{ and size of } s$.
 - estimated size of $r - s = \text{size of } r$.
 - All the three estimates may be quite inaccurate, but provide upper bounds on the sizes.



Estimation of Number of Distinct Values of Attribute A

Selections: $\sigma_{\theta}(r)$

- If θ forces A to take a specified value: $V(A, \sigma_{\theta}(r)) = 1$.
 - e.g., $A = 3$
- If θ forces A to take on one of a specified set of values:
 $V(A, \sigma_{\theta}(r)) = \text{number of specified values}$.
 - (e.g., $(A = 1 \vee A = 3 \vee A = 4)$),
- If the selection condition θ is of the form $A \text{ op } v$, where op is a comparison operator
estimated $V(A, \sigma_{\theta}(r)) = V(A.r) * s$
 - where s is the selectivity of the selection.
- In all the other cases: use approximate estimate of
 $\min(V(A, r), n_{\sigma_{\theta}(r)})$
 - More accurate estimate can be got using probability theory, but this one works fine generally



Estimation of Distinct Values (Cont.)

Joins: $r \bowtie s$

- If all attributes in A are from r
estimated $V(A, r \bowtie s) = \min (V(A, r), n_{r \bowtie s})$
- If A contains attributes $A1$ from r and $A2$ from s , then estimated $V(A, r \bowtie s) =$
$$\min(V(A1, r) * V(A2 - A1, s), V(A1 - A2, r) * V(A2, s), n_{r \bowtie s})$$
 - More accurate estimate can be got using probability theory, but this one works fine generally
- Estimation of distinct values are straightforward for projections.
 - They are the same in $\Pi_A(r)$ as in r .



Additional Optimization techniques



Optimizing Nested Subqueries**

- Nested query example:

```
select name  
from instructor  
where exists
```

```
    (select * from teaches
```

```
        where instructor.ID = teaches.ID and teaches.year = 2019)
```

- SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values
 - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause
 - Such evaluation is called **correlated evaluation**
 - Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery



Optimizing Nested Subqueries (Cont.)

- Correlated evaluation may be quite inefficient since
 - a large number of calls may be made to the nested query
 - there may be unnecessary random I/O as a result
- SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques
- E.g.,: earlier nested query can be rewritten in relational algebra as

$$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$$

- The two queries generate different numbers of duplicates (why?)
 - an instructor who teaches multiple sections in 2019 will appear multiple times in the result of the relational algebra query, although that instructor would appear only once in the SQL query result.
 - Can be modified to handle duplicates correctly using semijoins



Optimizing Nested Subqueries (Cont.)

- The **semijoin** operator $\bowtie$ is defined as follows
 - A tuple r_i appears n times in $r \bowtie_{\theta} s$ if it appears n times in r , and there is at least one matching tuple s_j in s

- E.g.: earlier nested query can be rewritten as

$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$

- Or even as:

$\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID} (\sigma_{teaches.year=2019} teaches))$

- Now the duplicate count is correct!

- The above relational algebra query is also equivalent to

select *name*

from *instructor*

where *ID* in (**select** *teaches.ID*

from *teaches*

where *teaches.year* = 2019)



Optimizing Nested Subqueries (Cont.)

- This could also be written using only joins (in SQL) as
with t_1 as
 (select distinct ID
 from $teaches$
 where $year = 2019$)

select $name$
from $instructor, t_1$
where $t_1.ID = instructor.ID$



Materialized Views

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view
create view *department_total_salary*(*dept_name*, *total_salary*) **as**
select *dept_name*, **sum**(*salary*)
from *instructor*
group by *dept_name*
- Materializing the above view would be very useful if the total salary by department is required frequently
 - Saves the effort of finding multiple tuples and adding up their amounts



Materialized View Maintenance

- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
 - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Incremental maintenance supported by many database systems
 - Avoids manual effort/correctness issues



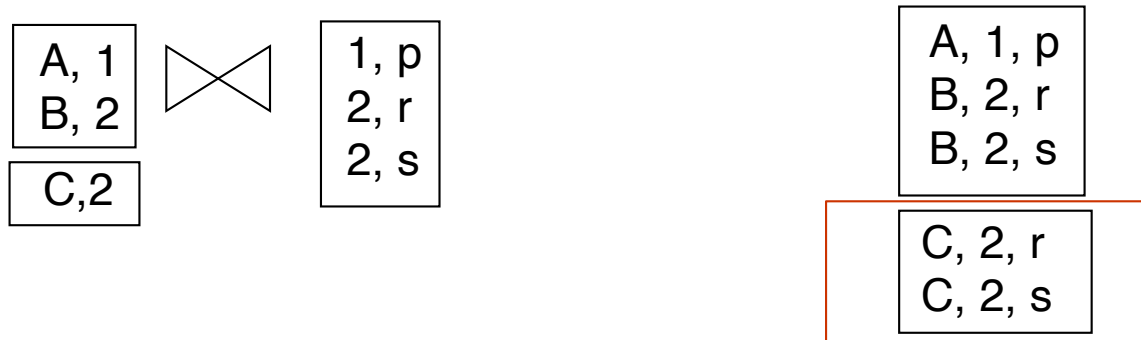
Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
 - Set of tuples inserted to and deleted from r are denoted i_r and d_r
- To simplify our description, we only consider inserts and deletes
 - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions



Join Operation

- Consider the materialized view $v = r \bowtie s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insertion to r :
 - We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
 - and rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
 - But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$





Selection and Projection Operations

- Selection: Consider a view $v = \sigma_{\theta}(r)$.
 - $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$ for inserts
 - $v^{new} = v^{old} - \sigma_{\theta}(d_r)$ for deletes
- Projection is a more difficult operation
 - $R = (A, B)$, and $r(R) = \{ (a, 2), (a, 3) \}$
 - $\Pi_A(r)$ has a single tuple (a) .
 - If we delete the tuple $(a, 2)$ from r , we should not delete the tuple (a) from $\Pi_A(r)$, but if we then delete $(a, 3)$ as well, we should delete the tuple
- For each tuple in a projection $\Pi_A(r)$, we will keep a count of how many times it was derived
 - On insert of a tuple to r , if the resultant tuple is already in $\Pi_A(r)$ we increment its count, else we add a new tuple with count = 1
 - On delete of a tuple from r , we decrement the count of the corresponding tuple in $\Pi_A(r)$
 - if the count becomes 0, we delete the tuple from $\Pi_A(r)$



Other Operations

- Set intersection: $v = r \cap s$
 - When a tuple is inserted into r , we check if it is present in s , and if so we add it to v .
 - If the tuple is deleted from r , we delete it from the intersection if it is present.
 - Updates to s are symmetric
 - The other set operations, *union* and *set difference* are handled in a similar fashion.



Query Optimization and Materialized Views

- Rewriting queries to use materialized views:
 - A materialized view $v = r \bowtie s$ is available
 - A user submits a query $r \bowtie s \bowtie t$
 - We can rewrite the query as $v \bowtie t$
 - Whether to do so depends on cost estimates for the two alternative
- Replacing the use of a materialized view by the view definition:
 - A materialized view $v = r \bowtie s$ is available, but without any index on it
 - User submits a query $\sigma_{A=10}(v)$.
 - Suppose also that s has an index on the common attribute B , and r has an index on attribute A .
 - The best plan for this query may be to replace v by $r \bowtie s$, which can lead to the query plan $\sigma_{A=10}(r) \bowtie s$
- Query optimizer should be extended to consider all above alternatives and choose the best overall plan



Top-K Queries

- **Top-K queries**

```
select *  
from r, s  
where r.B = s.B  
order by r.A ascending  
limit 10
```

- Alternative 1: Indexed nested loops join with r as outer
- Alternative 2: estimate highest r.A value H in result and add selection (**and** r.A <= H) to where clause
 - If < 10 results, retry with larger H



Optimization of Updates

■ Halloween problem

**update R set $A = 5 * A$
where $A > 10$**

- If index on A is used to find tuples satisfying $A > 10$, and tuples updated immediately, same tuple may be found (and updated) multiple times
- Solution 1: *Always defer updates*
 - collect the updates (old and new values of tuples) and update relation and indices in second pass
 - Drawback: extra overhead even if e.g. update is only on R.B, not on attributes in selection condition
- Solution 2: *Defer only if required*
 - Perform immediate update if update does not affect attributes in where clause, and deferred updates otherwise.



Join Minimization

- **Join minimization**

```
select r.A, r.B  
from r, s  
where r.B = s.B
```

- Check if join with s is redundant. If yes, drop it
 - E.g., join condition is on foreign key from r to s, r.B is declared as not null, and no selection on s



Multiquery Optimization

- Example

Q1: **select * from** (r natural join t) natural join s

Q2: **select * from** (r natural join u) natural join s

- Both queries share common subexpression (r natural join s)
- May be useful to compute (r natural join s) once and use it in both queries
 - But this may be more expensive in some situations
 - e.g. (r natural join s) may be expensive, plans as shown in queries may be cheaper

- **Multiquery optimization**: find best overall plan for a set of queries, exploiting sharing of common subexpressions between queries where it is useful



Multiquery Optimization (Cont.)

- Simple heuristic used in some database systems:
 - optimize each query separately
 - detect and exploiting common subexpressions in the individual optimal query plans
 - May not always give best plan, but is cheap to implement
 - **Shared scans**: widely used special case of multiquery optimization
 - particularly useful when multiple queries perform a scan on a single large relation (typically a “fact table”).
- Set of materialized views may share common subexpressions
 - As a result, view maintenance plans may share subexpressions
 - Multiquery optimization can be useful in such situations



Parametric Query Optimization

- Example
select *
from r natural join s
where r.a < \$1
 - value of parameter \$1 not known at compile time
 - known only at run time
 - different plans may be optimal for different values of \$1
- Solution 1: optimize at run time, each time query is submitted
 - can be expensive
- Solution 2: **Parametric Query Optimization:**
 - optimizer generates a set of plans, optimal for different values of \$1
 - Set of optimal plans usually small for 1 to 3 parameters
 - Key issue: how to do find set of optimal plans efficiently
 - best one from this set is chosen at run time when \$1 is known
- Solution 3: **Query Plan Caching**
 - If optimizer decides that same plan is likely to be optimal for all parameter values, it caches plan and reuses it, else reoptimize each time
 - Implemented in many database systems



Adaptive Query Processing

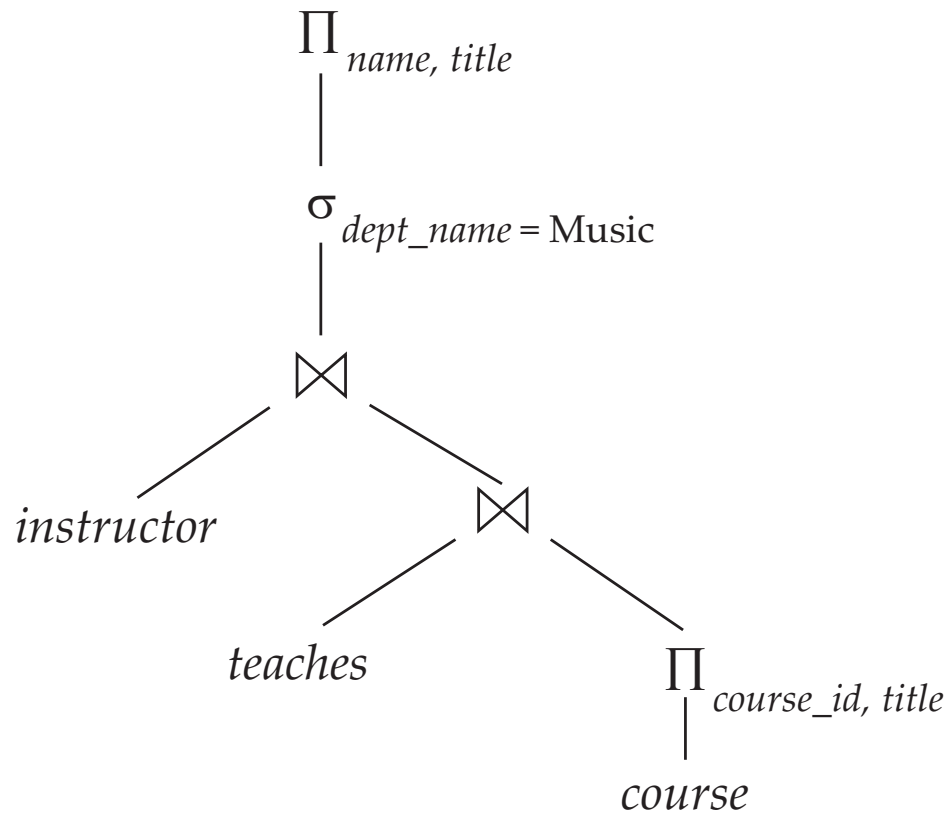
- Some systems support adaptive operators that change execution algorithm on the fly
 - E.g., (indexed) nested loops join or hash join chosen at run time, depending on size of outer input
- Other systems allow monitoring of behavior of plan at run time and adapt plan
 - E.g., if statistics such as number of rows is found to be very different in reality from what optimizer estimated
 - Can stop execution, compute fresh plan, and restart
 - But must avoid too many such restarts



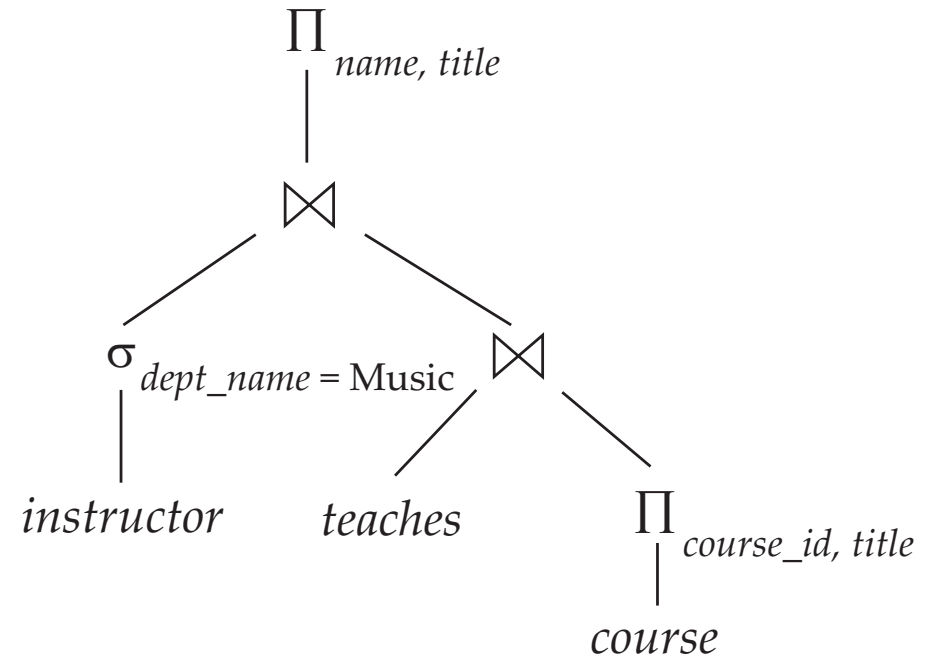
End of Chapter



Figure 13.01



(a) Initial expression tree



(b) Transformed expression tree



Figure 13.02

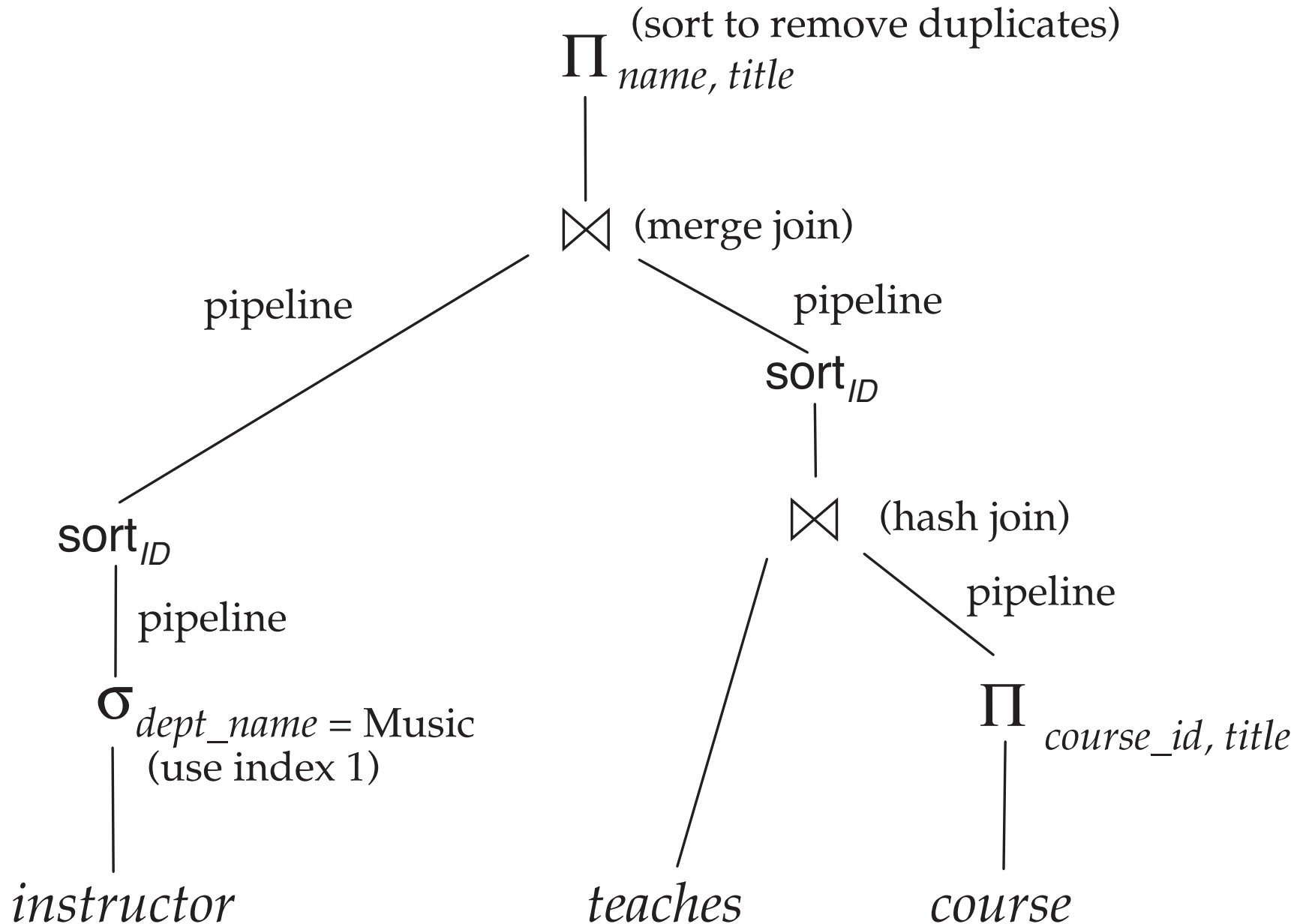
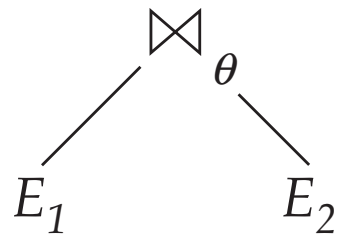
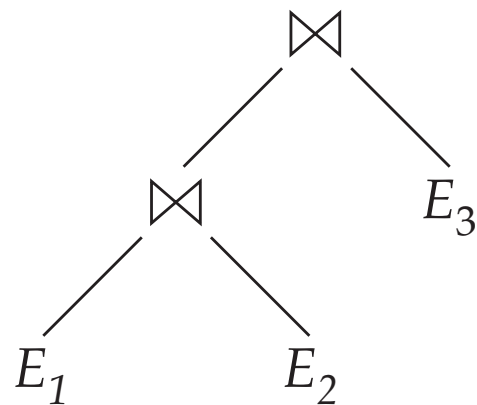
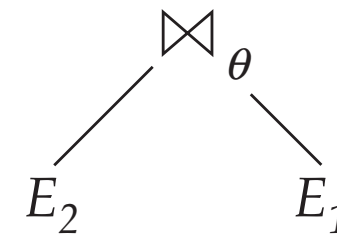




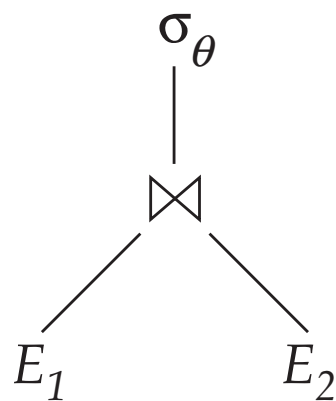
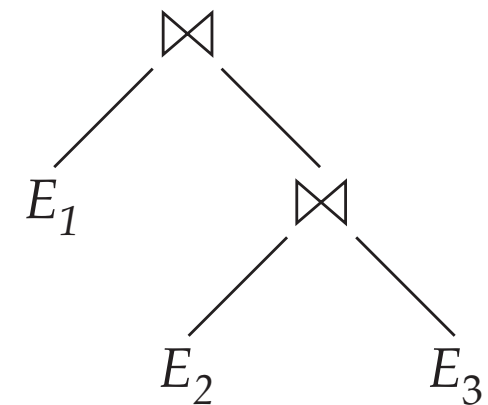
Figure 13.03



Rule 5



Rule 6.a



Rule 7.a

If θ only has attributes from E1

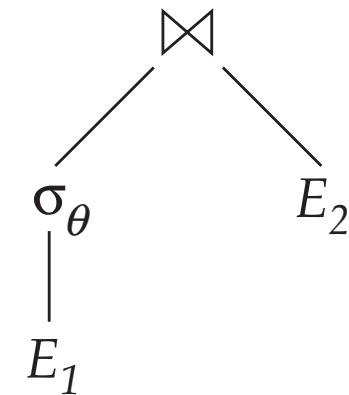
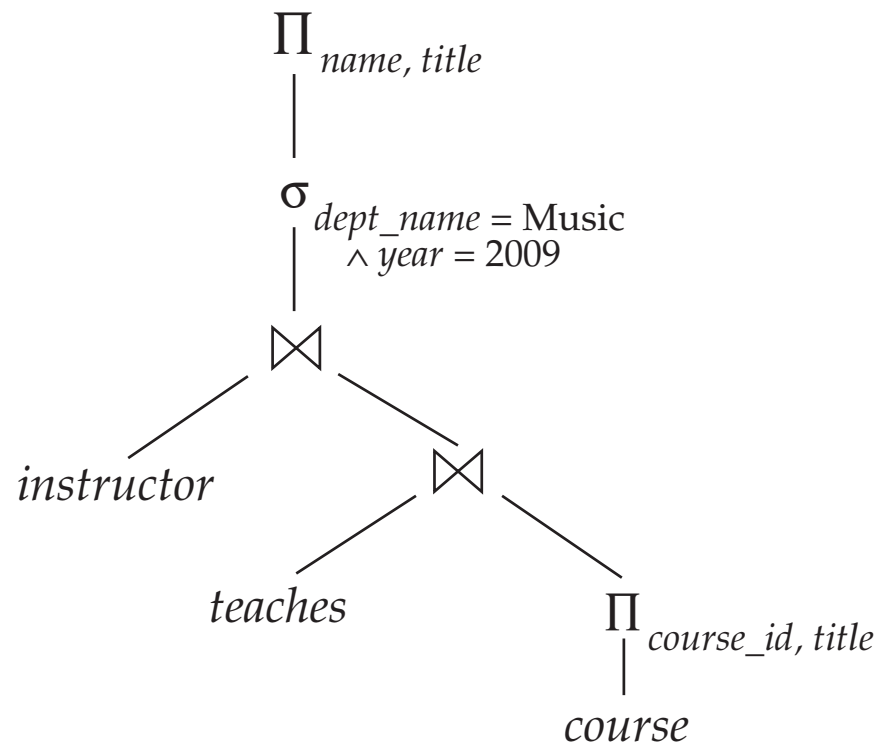
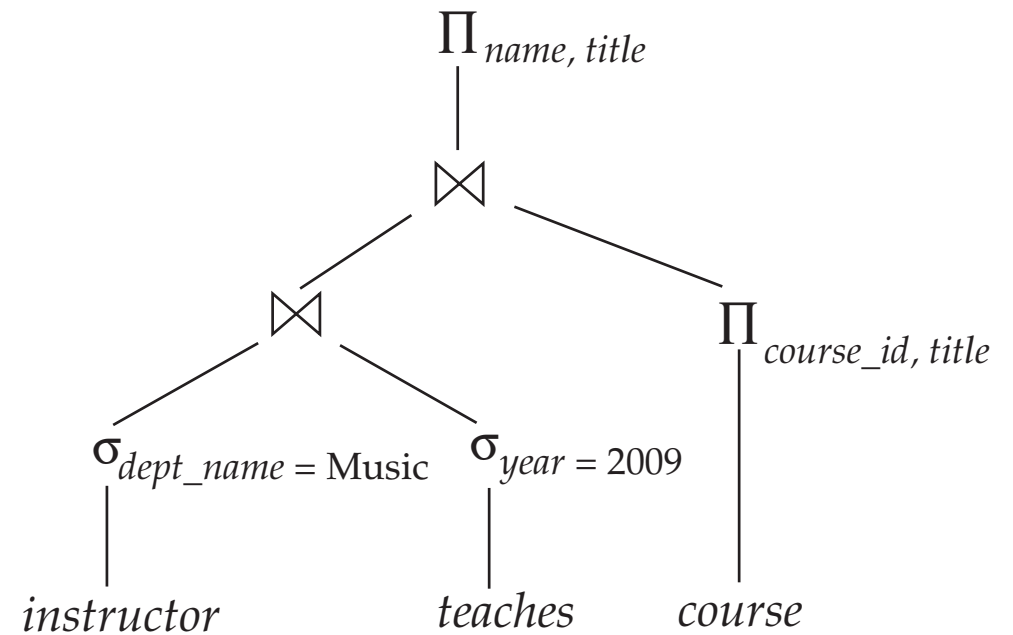




Figure 13.04



(a) Initial expression tree



(b) Tree after multiple transformations



Figure 13.06

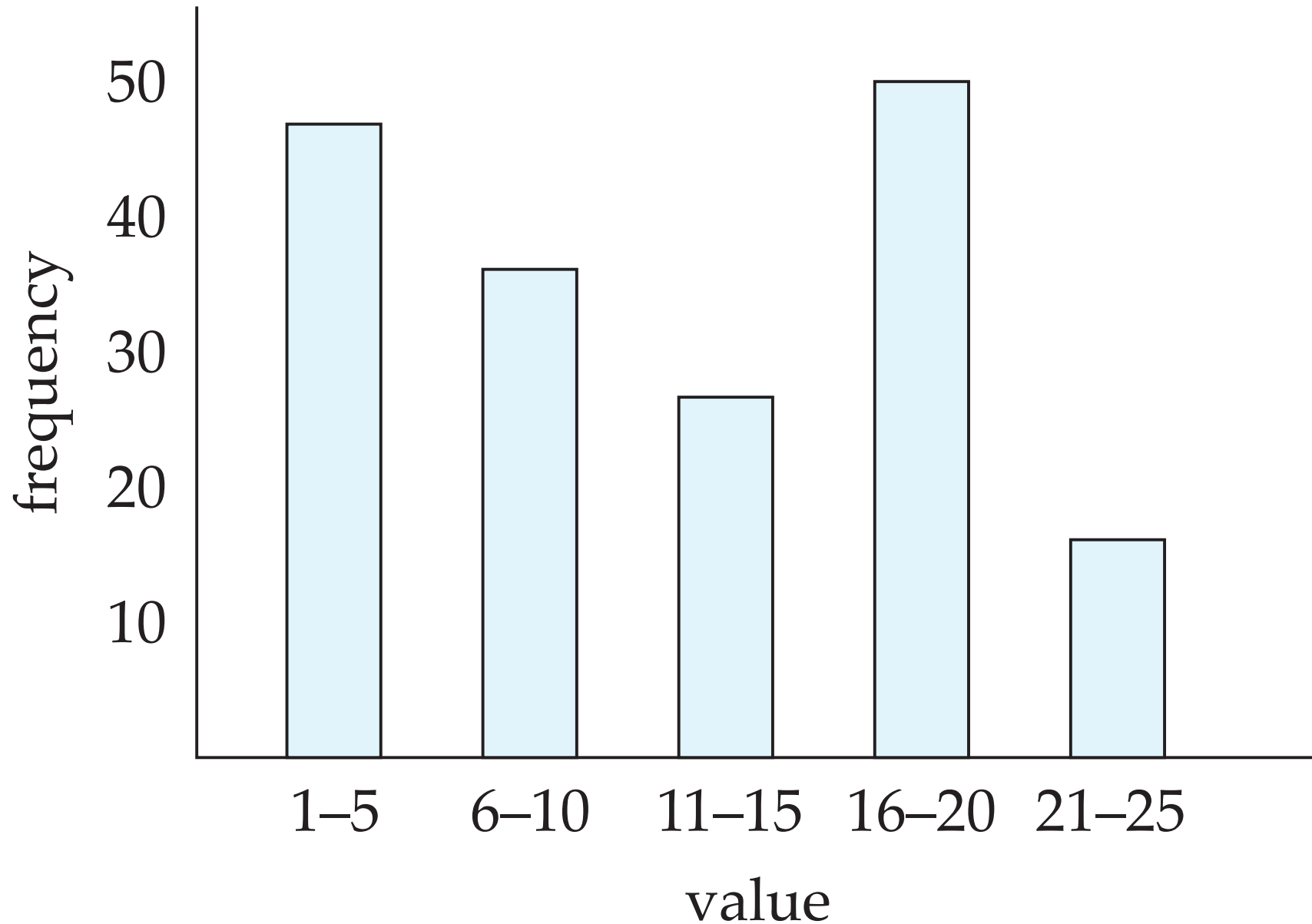
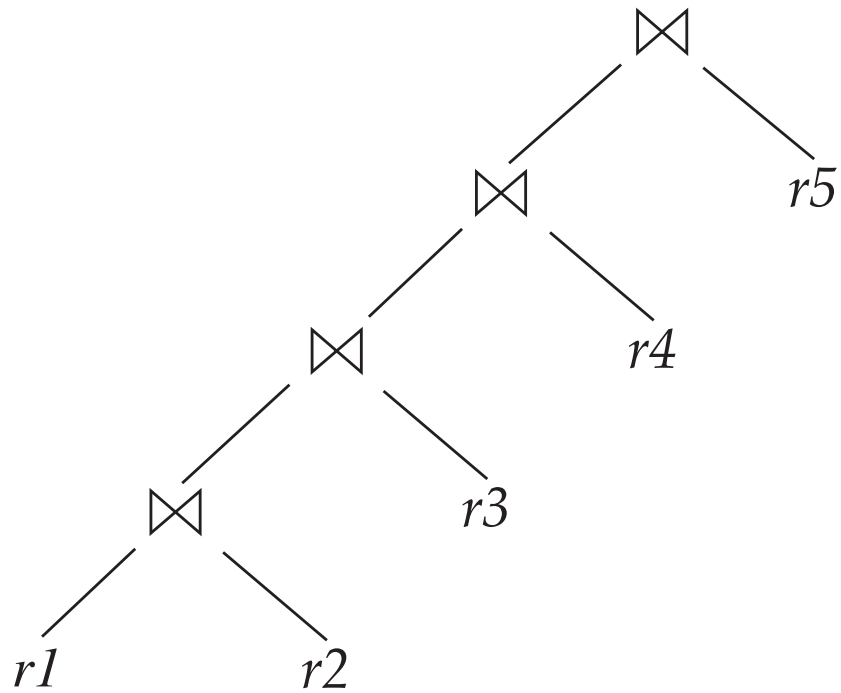
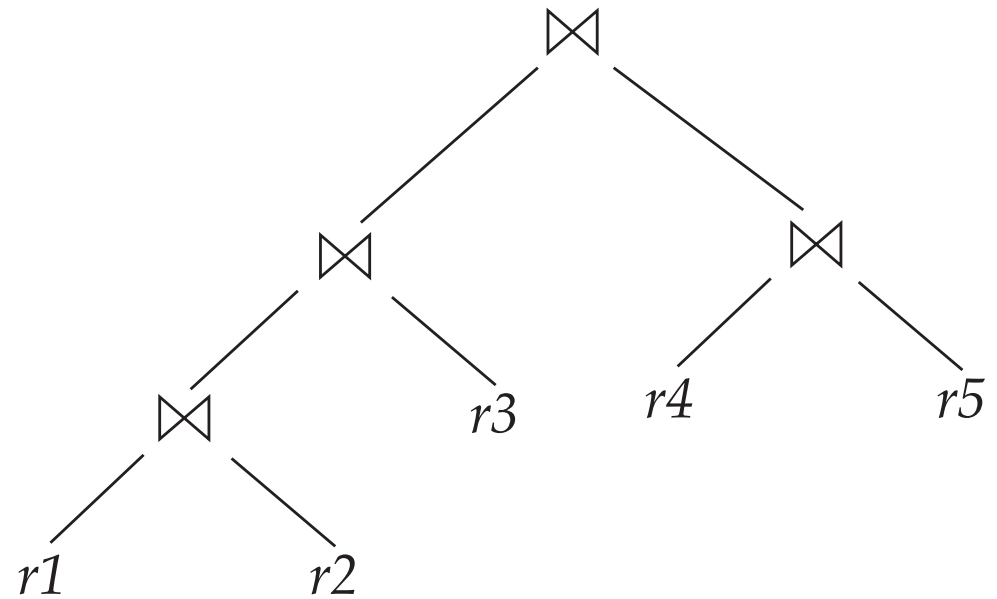




Figure 13.08



(a) Left-deep join tree



(b) Non-left-deep join tree